Lecture 11

GPU Architecture & GPU Programming with CUDA

Graphics Processing Units(GPU)

- Processing a single image can require very large amounts of data—hundreds of megabytes of data for a single image is not unusual.
- GPUs therefore need to maintain very high rates of data movement, and to avoid stalls on memory accesses, they rely heavily on hardware multithreading.
- GPUs can optimize performance by using SIMD parallelism, and in the current generation all GPUs use SIMD parallelism.
- This is obtained by including a large number of datapaths (e.g., 128) on each GPU processing core.

Cont.,

- Although the datapaths on a given core can use SIMD parallelism, current generation GPUs can run more than one instruction stream on a single core.
- Furthermore, typical GPUs can have dozens of cores, and these cores are also capable of executing independent instruction streams.
- So GPUs are neither purely SIMD nor purely MIMD.
- Also note that GPUs can use shared or distributed memory.
- The largest systems often use both.

GPU Architectures

- GPUs are composed of SIMD or Single Instruction stream, Multiple Data stream processors.
- So, to understand how to program them, we need to first look at their architecture.
- We can think of a SIMD processor as being composed of a single control unit and multiple datapaths.
- The control unit fetches an instruction from memory and broadcasts it to the datapaths.
- Each datapath either executes the instruction on its data or is idle

Example

- Suppose there are n datapaths that share an n-element array x.
- Also suppose that the ith datapath (core) will work with x[i].
- Now suppose we want to add 1 to the nonnegative elements of x and subtract 2 from the negative elements of x.
- We might implement this with the following code:

```
/ * Datapath i executes the following code * /
        i f ( x [ i ] >= 0) x [ i ] += 1;
        else x [ i ] −= 2;
```

By Using SIMD

- In a typical SIMD system, each datapath carries out the test

  x[i] >= 0.

- Then the datapaths for which the test is true execute x[i] += 1, while those for which x[i] < 0 are idle;
- Then the roles of the datapaths are reversed: those for which x[i] >= 0 are idle while the other datapaths execute x[i] −= 2.

Table 6.1 Execution of branch on a SIMD system.

Time	Datapaths with `x[i] >= 0`	Datapaths with `x[i] < 0`
1	Test `x[i] >= 0`	Test `x[i] >= 0`
2	`x[i] += 1`	Idle
3	Idle	`x[i] -= 2`

By Using GPU

- A typical GPU can be thought of as being composed of one or more SIMD processors.
- Nvidia GPUs are composed of Streaming Multiprocessors or SMs.
- One SM can have several control units and many more datapaths.
- So an SM can be thought of as consisting of one or more SIMD processors.
- The SMs, however, operate asynchronously: there is no penalty if one branch of an if−else executes on one SM, and the other executes on another SM.

Cont.,

- So, if all the threads with x[i] >= 0 were executing on one SM, and all the threads with x[i] < 0 were executing on another, the execution of our if−else example would require only two stages.

Table 6.2 Execution of branch on a system with multiple SMs.

Time	Datapaths with `x[i] >= 0` (on SM A)	Datapaths with `x[i] < 0` (on SM B)
1	Test `x[i] >= 0`	Test `x[i] >= 0`
2	`x[i] += 1`	`x[i] -= 2`

Nvidia Parlance

- In Nvidia parlance, the datapaths are called cores, Streaming (vector) Processors, or SPs.
- Currently, one of the most powerful Nvidia processor has 82 SMs, and each SM has 128 SPs for a total of 10,496 SPs.
- Also note that Nvidia uses the term SIMT instead of SIMD.
- SIMT stands for Single Instruction Multiple Thread, and the term is used because threads on an SM that are executing the same instruction may not execute simultaneously.
- To hide memory access latency, some threads may block while memory is accessed and other threads, that have already accessed the data, may proceed with execution.

Cont.,

- Each SM has a relatively small block of memory that is shared among its SPs.
- As we'll see, this memory can be accessed very quickly by the SPs.
- All of the SMs on a single chip also have access to a much larger block of memory that is shared among all the SPs.
- Accessing this memory is relatively slow. (See Fig. 6.1.)

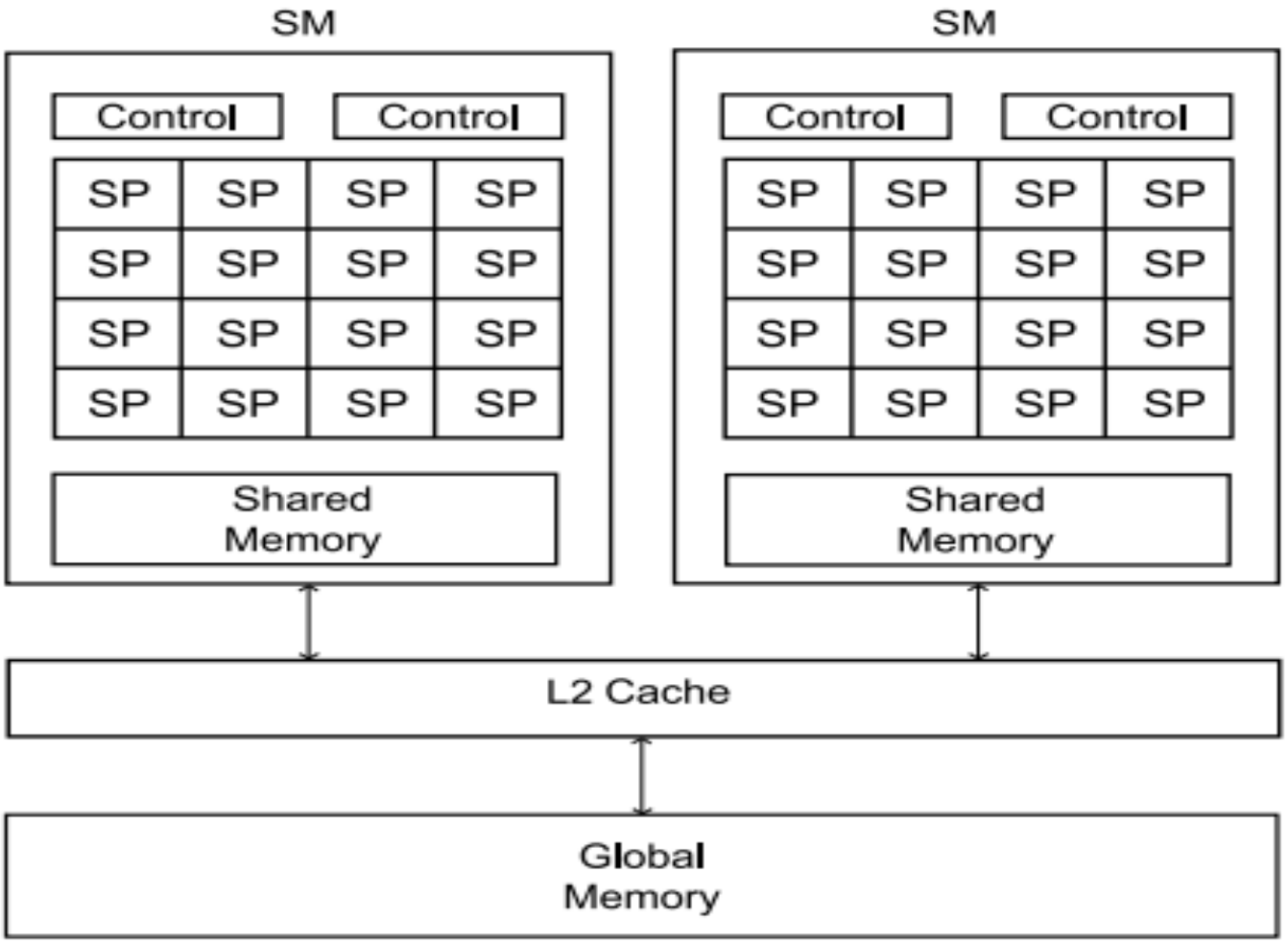

FIGURE 6.1

Simplified block diagram of a GPU.

Cont.,

- The GPU and its associated memory are usually physically separate from the CPU (Host) and its associated memory.
- In Nvidia documentation, the CPU together with its associated memory is often called the host, and the GPU together with its memory is called the device.
- In earlier systems the physical separation of host and device memories required that data was usually explicitly transferred between CPU memory and GPU memory.
- That is, a function was called that would transfer a block of data from host memory to device memory or vice versa.

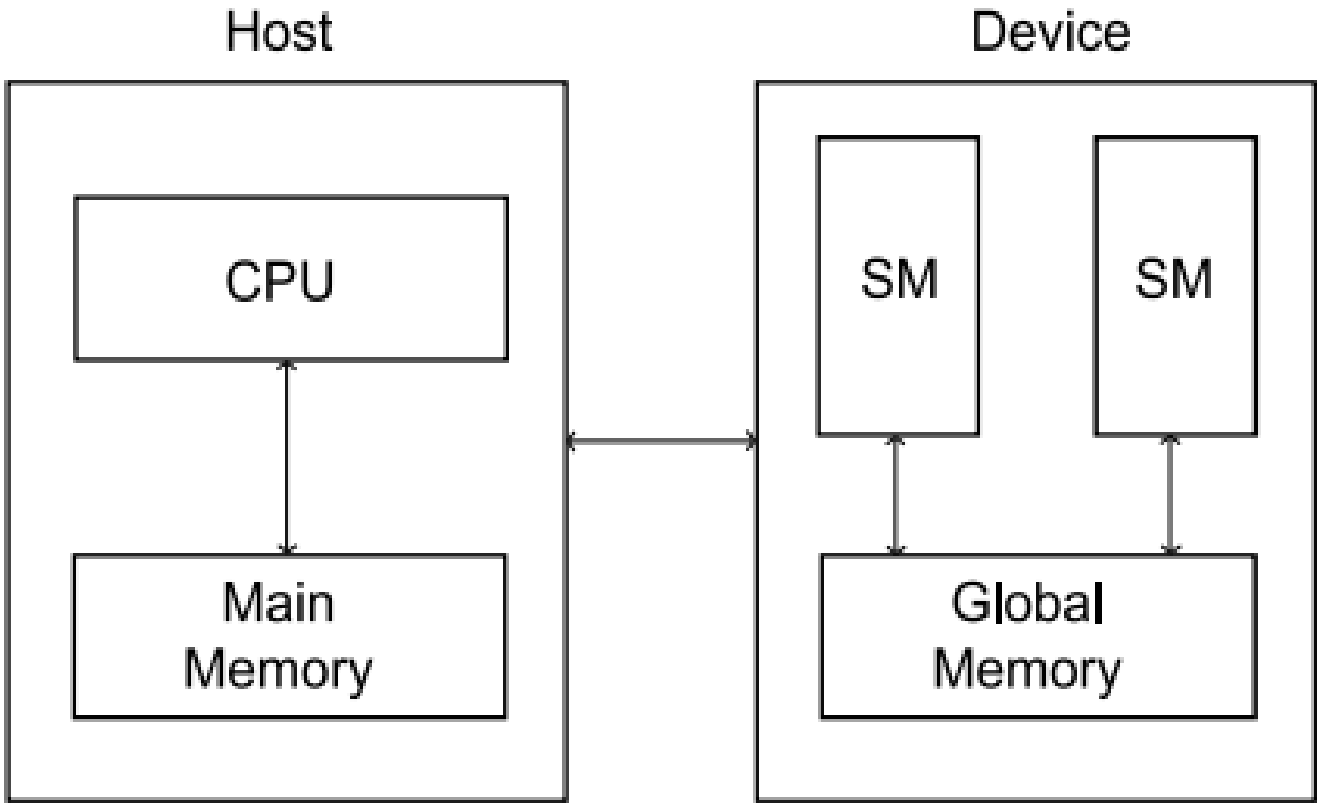

FIGURE 6.2

Simplified block diagram of a CPU and a GPU.

Compute Capability

- It is a number assigned by NVIDIA to each GPU model to represent its level of features and programming support.
- There are limits on the number of threads and the number of blocks depend on what Nvidia calls the compute capability of the GPU.
- GPUs with compute capability ≥ 3.0 are generally considered powerful enough to run modern parallel computing workloads, such as those for scientific research, deep learning, data analysis, and more.
- In GPUs with compute capability < 3.0), when writing CUDA code, the programmer had to manually manage the transfer of data between the **host (CPU)** and the **device (GPU)**.

Note:

- This would require explicit memory copying commands in the code, like cudaMemcpy(), to move data between CPU and GPU memory.
- With newer GPUs (compute capability ≥ 3.0), Nvidia introduced improvements in hardware and software that allow for **implicit memory management** in certain cases.
- This means that you don't always need to manually manage data transfers in the source code for the program to run correctly.

Unified Memory

- For example, in modern GPUs, features like **Unified Memory** (introduced with the Kepler architecture) allow both the CPU and GPU to access a shared memory space.
- The underlying system(which is the HW & SW infrastructure that support parallel computing) automatically handles data movement between the two (CPU & GPU) without requiring **explicit** commands in the code, reducing the programmer's workload.
- This made Kepler GPUs particularly suitable for high-performance computing (HPC) tasks, scientific simulations, and deep learning.

Dynamic Parallelism

- **Dynamic Parallelism** is a powerful feature introduced with Nvidia's **Kepler architecture** that enables the GPU to launch new **kernels** (GPU functions) directly from within a running kernel.
- This ability to launch additional kernels without having to return control to the CPU provides a significant boost in flexibility and performance, especially for complex algorithms that require recursive or nested parallelism.

How Dynamic Parallelism Works:

- In traditional GPU programming (before Kepler), if you wanted to launch a new kernel (a set of instructions for the GPU to execute), you had to do so from the CPU.
- **The flow would look like this:**

1. The CPU prepares the data and launches a kernel on the GPU.
2. Once the GPU finishes executing that kernel, it would return control to the CPU.
3. The CPU would then process the results and potentially launch another kernel if needed.

- However, **with Dynamic Parallelism**, the GPU itself can directly launch additional kernels from within an already executing kernel, removing the need to return to the CPU.
- This makes it much easier to handle recursive tasks and workloads that involve a variable number of parallel tasks.

Hyper-Q (Hyper-Queue)

- Hyper-Q allows for more efficient use of the GPU's resources, particularly in terms of concurrent processing.
- Hyper-Q is designed to enable **multiple CPU threads or CUDA streams** to concurrently submit work to a single GPU without significant contention, rather than serially.
- Before Hyper-Q (e.g., in the Fermi architecture), the GPU could only accept work from **one stream at a time**.
- This created a bottleneck where even if there were idle GPU resources, they couldn't be used efficiently unless the single stream was always providing work.

Hyper-Q solves this by:

- Increasing the number of hardware work queues from **1 to 32**
- Allowing **multiple CPU cores** or **CUDA streams** to feed work to the GPU simultaneously
- Reducing **false dependencies**, where work is artificially serialized.
- This is especially useful in scenarios where many tasks need to be processed simultaneously, such as in high-performance computing (HPC), scientific simulations, or large-scale data processing.

Example: Without & With Hyper-Q

- **Without:**
- [Thread 1] ----\
- [Thread 2] -----+---> [SINGLE GPU WORK QUEUE] -- -> GPU
- [Thread 3] ----/
- **With:**
- [Thread 1] ---> [GPU Work Queue 1] \
- [Thread 2] ---> [GPU Work Queue 2] \
- [Thread 3] ---> [GPU Work Queue 3] ---> GPU (Parallel execution)
- ...
- [Thread 32]--> [GPU Work Queue 32] /

GPU and GPGPU

- In the late 1990s and early 2000s, the computer industry responded to the demand for highly realistic computer video games and video animations by developing extremely powerful graphics processing units or GPUs.
- These processors, as their name suggests, are designed to improve the performance of programs that need to render many detailed images.
- By the early 2000s they were trying to apply the power of GPUs to solving general computational problems, problems such as searching and sorting, rather than graphics.
- This became known as General Purpose computing on GPUs or GPGPU.

Cont.,

- One of the biggest difficulties faced by the early developers of GPGPU was that the GPUs of the time could only be programmed using computer graphics APIs.
- Direct3D and OpenGL (Open Graphics Library) are graphics APIs used to render 2D and 3D vector graphics, and it's commonly used in video games, simulations, CAD applications, and other visual software.
- Several groups started work on developing languages and compilers that allowed programmers to implement general algorithms for GPUs in APIs that more closely resembled conventional, high-level languages for CPUs.

Heterogeneous Computing

- Writing a program that runs on a GPU is an example of heterogeneous computing.
- The reason is that the programs make use of both a host processor—a conventional CPU—and a device processor—a GPU—and, as we've just seen, the two processors have different architectures.
- We'll still write a single program (using the SPMD approach), but now there will be functions that we write for conventional CPUs and functions that we write for GPUs.
- So, effectively, we'll be writing two programs.

GPGPU API's

- These efforts led to the development of several APIs for general purpose programming on GPUs.
- Currently the most widely used APIs are CUDA and OpenCL.
- CUDA (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA.
- OpenCL (Open Computing Language) is an open standard for parallel programming that allows developers to write programs that can run across a variety of computing platforms, such as CPUs, GPUs, and even other processors like FPGAs, and DSPs.

CUDA

- CUDA is a software platform that can be used to write GPGPU programs for heterogeneous systems equipped with an Nvidia GPU.
- CUDA was originally an acronym for “Compute Unified Device Architecture,” which was meant to suggest that it provided a single interface for programming both CPU and GPU.
- More recently, however, Nvidia has decided that CUDA is not an acronym; it’s simply the name of an API for GPGPU programming.

Cont.,

- There is a language-specific CUDA API for several languages; for example, there are CUDA APIs for C, C++, Fortran, Python, and Java.
- We'll be using CUDA C, but we need to be aware that sometimes we'll need to use some C++ constructs.
- This is because the CUDA C compiler can compile both C and C++ programs, and it can do this because it is a modified C++ compiler.
- So where the specifications for C and C++ differ the CUDA compiler sometimes uses C++.

CUDA ‘Hello’ Program

- As usual, we’ll begin by implementing a version of the “hello, world” program.
- We’ll write a CUDA C program in which each CUDA thread prints a greeting.
- Since the program is heterogeneous, we will, effectively, be writing two programs: a host or CPU program and a device or GPU program.
- Note that even though our programs are written in CUDA C, CUDA programs cannot be compiled with an ordinary C compiler.

Cont.,

- So unlike MPI and Pthreads, CUDA is not just a library that can be linked in to an ordinary C program: CUDA requires a special compiler.
- For example, an ordinary C compiler (such as gcc) generates a machine language executable for a single CPU (e.g., an x86 processor), but the CUDA compiler must generate machine language for two different processors: the host processor and the device processor.

The Source Code

- The source code for a CUDA program that prints a greeting from each thread on the GPU is shown in Program 6.1.
- As you might guess, there's a header file for CUDA programs, which we include in Line 2.
- The Hello function follows the include directives and starts on Line 5.
- This function is run by each thread on the GPU.
- In CUDA parlance, it's called a kernel, a function that is started by the host but runs on the device.
- CUDA kernels are identified by the keyword __global__, and they always have return type void.

```
1  #include <stdio.h>
2  #include <cuda.h>    /* Header file for CUDA */
3
4  /* Device code:  runs on GPU */
5  __global__ void Hello(void) {
6
7      printf("Hello from thread %d!\n", threadIdx.x);
8  }  /* Hello */
9
10
11 /* Host code:  Runs on CPU */
12 int main(int argc, char* argv[]) {
13     int thread_count;       /* Number of threads to run on GPU */
14
15     thread_count = strtol(argv[1], NULL, 10);
16                         /* Get thread_count from command line */
17
18     Hello <<<1, thread_count>>>();
19                         /* Start thread_count threads on GPU, */
20
21     cudaDeviceSynchronize();          /* Wait for GPU to finish */
22
23     return 0;
24 }  /* main */
```

Program 6.1: CUDA program that prints greetings from the threads.

Where:

- Like ordinary C programs, CUDA C programs start execution in main, and the main function runs on the host.
- The function first gets the number of threads from the command line.
- Things get interesting in the call to the kernel in Line 18.
- Here we tell the system how many threads to start on the GPU by enclosing the pair 1 , thread_count in triple angle brackets.
- If there were any arguments to the Hello function, we would enclose them in the following parentheses.

Cont.,

- The call to cudaDeviceSynchronize will cause the main program to wait until all the threads have finished executing the kernel, and when this happens, the program terminates as usual with return 0.
- A CUDA program file that contains both host code and device code should be stored in a file with a ".cu" suffix.
- For example, our hello program is in a file called cuda_hello.cu.
- We can compile it using the CUDA compiler nvcc.

Cont.,

- The decimal int format specifier (%d) refers to the variable threadIdx.x.
- The struct threadIdx is one of several variables defined by CUDA when a kernel is started.
- When the kernel is started, the struct threadIdx is initialized by the system, and in our example the field threadIdx.x contains the thread's index or rank.
- So we use it to print a message containing the thread's rank.
- After a thread has printed its message, it terminates execution.
- In this case, the only thread-specific data is the thread rank stored in threadIdx.x.

kernel execution is asynchronous

- One very important difference between the execution of an ordinary C function and a CUDA kernel is that kernel execution is asynchronous.
- This means that when a kernel (a function that runs on the GPU) is launched from the host (CPU), the function call **returns immediately** without waiting for the kernel to finish executing.
- Allows the CPU to **do other work** while the GPU is executing.
- When you call a kernel like *myKernel*<<<grid, block>>>(args); in CUDA, it **returns control to the CPU immediately**.
- The GPU begins executing the kernel in parallel, but the host doesn't wait for it to finish.
- To ensure the kernel has completed, you must **explicitly synchronize**: In **CUDA**: cudaDeviceSynchronize();

Threads, Blocks, and Grids

- You're probably wondering why we put a "1" in the angle brackets in our call to Hello: Hello <<<1, thread_count>>>();
- Recall that an Nvidia GPU consists of a collection of streaming multiprocessors (SMs), and each streaming multiprocessor consists of a collection of streaming processors (SPs).
- When a CUDA kernel runs, each individual thread will execute its code on an SP.
- With "1" as the first value in angle brackets, all of the threads that are started by the kernel call will run on a single SM.
- If our GPU has two SMs, we can try to use both of them with the kernel call: Hello <<<2, thread_count/2>>>();

Blocks

- A thread block is a collection of threads that run on a single SM.
- In a kernel call the first value in the angle brackets specifies the number of thread blocks.
- The second value is the number of threads in each thread block.
- So when we started the kernel with:

```
Hello <<< 1, thread_count>>>();
```

- We were using one thread block, which consisted of thread_count threads, and, as a consequence, we only used one SM.

Grids

- A grid is just the collection of thread blocks started by a kernel.
- So a thread block is composed of threads, and a grid is composed of thread blocks.
- There are several built-in variables that a thread can use to get information on the grid started by the kernel.
- The following four variables are structs that are initialized in each thread's memory when a kernel begins execution:

1. threadIdx: the rank or index of the thread in its thread block.
2. blockDim: the dimensions, shape, or size of the thread blocks.
3. blockIdx: the rank or index of the block within the grid.
4. gridDim: the dimensions, shape, or size of the grid.

Cont.,

- All of these structs have three fields, x, y, and z, and the fields all have unsigned integer types.
- The fields are often convenient for applications.
- For example, an application that uses graphics may find it convenient to assign a thread to a point in two- or three-dimensional space, and the fields in threadIdx can be used to indicate the point's position.
- An application that makes extensive use of matrices may find it convenient to assign a thread to an element of a matrix, and the fields in threadIdx can be used to indicate the column and row of the element.

When we call a kernel with something like

```
int blk_ct, th_per_blk;
...
Hello <<<blk_ct, th_per_blk>>>();
```

the three-element structures `gridDim` and `blockDim` are initialized by assigning the values in angle brackets to the `x` fields. So, effectively, the following assignments are made:

```
gridDim.x = blk_ct;
blockDim.x = th_per_blk;
```

The `y` and `z` fields are initialized to 1. If we want to use values other than 1 for the `y` and `z` fields, we should declare two variables of type `dim3`, and pass them into the call to the kernel. For example,

```
dim3 grid_dims, block_dims;
grid_dims.x = 2;
grid_dims.y = 3;
grid_dims.z = 1;
block_dims.x = 4;
block_dims.y = 4;
block_dims.z = 4;
...
Kernel <<<grid_dims, block_dims>>> (...);
```

This should start a grid with $2 \times 3 \times 1 = 6$ blocks, each of which has $4^3 = 64$ threads.

Note:

- Note that all the blocks must have the same dimensions.
- More importantly, CUDA requires that thread blocks be independent.
- So one thread block must be able to complete its execution, regardless of the states of the other thread blocks.
- The thread blocks can be executed sequentially in any order, or they can be executed in parallel.

Modified ‘Hello’ Program

- We can modify our greetings program so that it uses a user-specified number of blocks, each consisting of a user-specified number of threads. (See Program 6.2.)
- In this program we get both the number of thread blocks and the number of threads in each block from the command line.
- Now the kernel call starts blk_ct thread blocks, each of which contains th_per_blk threads.
- When the kernel is started, each block is assigned to an SM, and the threads in the block are then run on that SM.

Cont.,

- The output is similar to the output from the original program, except that now we're using two system-defined variables: threadIdx.x and blockIdx.x.
- As you've probably guessed, threadIdx.x gives a thread's rank or index in its block, and blockIdx.x gives a block's rank in the grid.

```
1  #include <stdio.h>
2  #include <cuda.h>    /* Header file for CUDA */
3
4  /* Device code:  runs on GPU */
5  __global__ void Hello(void) {
6
7      printf("Hello from thread %d in block %d\n",
8             threadIdx.x, blockIdx.x);
9  }  /* Hello */
10
11
12 /* Host code:  Runs on CPU */
13 int main(int argc, char* argv[]) {
14     int blk_ct;                    /* Number of thread blocks */
15     int th_per_blk;       /* Number of threads in each block */
16
17     blk_ct = strtol(argv[1], NULL, 10);
18                     /* Get number of blocks from command line */
19     th_per_blk = strtol(argv[2], NULL, 10);
20         /* Get number of threads per block from command line */
21
22     Hello <<<blk_ct, th_per_blk>>>();
23               /* Start blk_ct*th_per_blk threads on GPU, */
24
25     cudaDeviceSynchronize();          /* Wait for GPU to finish */
26
27     return 0;
28 }  /* main */
```

Program 6.2: CUDA program that prints greetings from threads in multiple blocks.

Example: Vector Addition

- GPUs and CUDA are designed to be especially effective when they run data-parallel programs.
- So let's write a very simple, data-parallel CUDA program that's embarrassingly parallel: a program that adds two vectors or arrays.
- We'll define three n-element arrays, x, y, and z.
- We'll initialize x and y on the host.
- Then a kernel can start at least n threads, and the ith thread will add: z [i] = x [i] + y [i] ;
- Since GPUs tend to have more 32-bit than 64-bit floating point units, let's use arrays of floats rather than doubles:

  float *x , *y , *z ;

Cont.,

- After allocating and initializing the arrays, we'll call the kernel, and after the kernel completes execution, the program checks the result, frees memory, and quits.
- See Program 6.3, which shows the kernel and the main function.

```
1  __global__ void Vec_add(
2          const float x[]   /* in  */,
3          const float y[]   /* in  */,
4          float       z[]   /* out */,
5          const int   n     /* in  */) {
6      int my_elt = blockDim.x * blockIdx.x + threadIdx.x;
7
8      /* total threads = blk_ct*th_per_blk may be > n */
9      if (my_elt < n)
10         z[my_elt] = x[my_elt] + y[my_elt];
11 }  /* Vec_add */
12
13 int main(int argc, char* argv[]) {
14     int n, th_per_blk, blk_ct;
15     char i_g;  /* Are x and y user input or random? */
16     float *x, *y, *z, *cz;
17     double diff_norm;
18
19     /* Get the command line arguments, and set up vectors */
20     Get_args(argc, argv, &n, &blk_ct, &th_per_blk, &i_g);
21     Allocate_vectors(&x, &y, &z, &cz, n);
22     Init_vectors(x, y, n, i_g);
23
24     /* Invoke kernel and wait for it to complete          */
25     Vec_add <<<blk_ct, th_per_blk>>>(x, y, z, n);
26     cudaDeviceSynchronize();
27
28     /* Check for correctness */
29     Serial_vec_add(x, y, cz, n);
30     diff_norm = Two_norm_diff(z, cz, n);
31     printf("Two-norm of difference between host and ");
32     printf("device = %e\n", diff_norm);
33
34     /* Free storage and quit */
35     Free_vectors(x, y, z, cz);
36     return 0;
37 }  /* main */
```

Program 6.3: Kernel and `main` function of a CUDA program that adds two vectors.

The kernel

- We can view the CUDA kernel as taking the serial for loop and assigning each iteration to a different thread.
- This is often how we start the design process when we want to parallelize a serial code for CUDA: assign the iterations of a loop to individual threads.
- In the kernel (Lines 1–11), we first determine which element of z the thread should compute.
- We've chosen to make this index the same as the global rank or index of the thread.
- Since we're only using the x fields of the blockDim and threadIdx structs, there are a total of threads =

$$\text{gridDim . x} * \text{blockDim . X}$$

Cont.,

- So we can assign a unique “global” rank or index to each thread by using the formula:

$$rank = blockDim . x * blockIdx . x + threadIdx . x$$

- For example, if we have four blocks and five threads in each block, then the global ranks or indexes are shown in Table 6.4.

Table 6.4 Global thread ranks or indexes in a grid with 4 blocks and 5 threads per block.

	threadIdx.x				
blockIdx.x	**0**	**1**	**2**	**3**	**4**
0	0	1	2	3	4
1	5	6	7	8	9
2	10	11	12	13	14
3	15	16	17	18	19

Cont.,

- In the kernel, we assign this global rank to my_elt and use this as the subscript for accessing each thread's elements of the arrays x, y, and z.
- Note that we've allowed for the possibility that the total number of threads may not be exactly the same as the number of components of the vectors.
- So before carrying out the addition:

```
z [ my_elt ] = x [ my_elt ] + y [ my_elt ] ;
```

we first check that my_elt < n.

Get_args

- After declaring the variables, the main function calls a Get_args function, which returns n, the number of elements in the arrays, blk_ct, the number of thread blocks, and th_per_blk, the number of threads in each block.
- It also returns a char i_g (i : input manully, g: generate randomly).
- This tells the program whether the user will input x and y or whether it should generate them using a random number generator.
- If the user doesn't enter the correct number of command line arguments, the function prints a usage summary and terminates execution.

Cont.,

- Also if n is greater than the total number of threads, it prints a message and terminates. (See Program 6.5.)
- Note that Get_args is written in standard C, and it runs completely on the host.

```
1  void Get_args(
2        const int argc             /* in  */,
3        char*     argv[]           /* in  */,
4        int*      n_p              /* out */,
5        int*      blk_ct_p         /* out */,
6        int*      th_per_blk_p     /* out */,
7        char*     i_g              /* out */) {
8     if (argc != 5) {
9        /* Print an error message and exit */
10       ...
11    }
12
13    *n_p = strtol(argv[1], NULL, 10);
14    *blk_ct_p = strtol(argv[2], NULL, 10);
15    *th_per_blk_p = strtol(argv[3], NULL, 10);
16    *i_g = argv[4][0];
17
18    /* Is n > total thread count = blk_ct*th_per_blk? */
19    if (*n_p > (*blk_ct_p)*(*th_per_blk_p)) {
20       /* Print an error message and exit */
21       ...
22    }
23 }  /* Get_args */
```

Program 6.5: `Get_args` function from CUDA program that adds two vectors.

Allocate_vectors and managed memory

- After getting the command line arguments, the main function calls Allocate_vectors, which allocates storage for four n-element arrays of float : x , y , z , cz.
- The first three arrays are used on both the host and the device (Unified memory).
- The fourth array, cz, is only used on the host: we use it to compute the vector sum with one core of the host.
- We do this so that we can check the result computed on the device. (See Program 6.6.)

```
1  void Allocate_vectors(
2        float** x_p     /* out */,
3        float** y_p     /* out */,
4        float** z_p     /* out */,
5        float** cz_p    /* out */,
6        int     n       /* in  */) {
7
8     /* x, y, and z are used on host and device */
9     cudaMallocManaged(x_p, n*sizeof(float));
10    cudaMallocManaged(y_p, n*sizeof(float));
11    cudaMallocManaged(z_p, n*sizeof(float));
12
13    /* cz is only used on host */
14    *cz_p = (float*) malloc(n*sizeof(float));
15 }  /* Allocate_vectors */
```

Program 6.6: Array allocation function of CUDA program that adds two vectors.

Other functions called from main

- The function Init_vectors either reads x and y from stdin using scanf or generates them using the C library function random.
- It uses the last command line argument i_g to decide which it should do.
- The Serial_vec_add function (Program 6.4) just adds x and y on the host using a for loop.
- It stores the result in the host array cz.

```
1  void Serial_vec_add(
2        const float  x[]   /* in  */,
3        const float  y[]   /* in  */,
4        float        cz[]  /* out */,
5        const int    n     /* in  */) {
6
7     for (int i = 0; i < n; i++)
8        cz[i] = x[i] + y[i];
9  }  /* Serial_vec_add */
```

Program 6.4: Serial vector addition function.

Cont.,

- The Two_norm_diff function computes the "distance" between the vector z computed by the kernel and the vector cz computed by Serial_vec_add.
- So it takes the difference between corresponding components of z and cz, squares them, adds the squares, and takes the square root:

$$\sqrt{(z[0]-cz[0])^2+(z[1]-cz[1])^2+\cdots+(z[n-1]-cz[n-1])^2}.$$

See Program 6.7.

```
1  double Two_norm_diff(
2        const float  z[]   /* in */,
3        const float  cz[]  /* in */,
4        const int    n     /* in */) {
5     double diff, sum = 0.0;
6
7     for (int i = 0; i < n; i++) {
8        diff = z[i] - cz[i];
9        sum += diff*diff;
10    }
11    return sqrt(sum);
12 }  /* Two_norm_diff */
```

Program 6.7: C function that finds the distance between two vectors.

Cont.,

- The Free_vectors function just frees the arrays allocated by Allocate_vectors.
- The array cz is freed using the C library function free, but since the other arrays are allocated using cudaMallocManaged, they must be freed by calling:

  cudaFree: __host__ __device__ cudaError_t cudaFree (void * ptr)

- The qualifier __device__ is a CUDA addition to C, and it indicates that the function can be called from the device.
- So **cudaFree** can be called from the host or the device.
- However, if a pointer is allocated on the device, it cannot be freed on the host, and vice versa.

```
1  void Free_vectors(
2          float*  x     /* in/out */,
3          float*  y     /* in/out */,
4          float*  z     /* in/out */,
5          float*  cz    /* in/out */) {
6
7      /* Allocated with cudaMallocManaged */
8      cudaFree(x);
9      cudaFree(y);
10     cudaFree(z);
11
12     /* Allocated with malloc */
13     free(cz);
14 }   /* Free_vectors */
```

Program 6.8: CUDA function that frees four arrays.

Assignments

- How to modify the vector addition program for a system that doesn’t provide unified memory?
- Write CUDA trapezoidal rule?